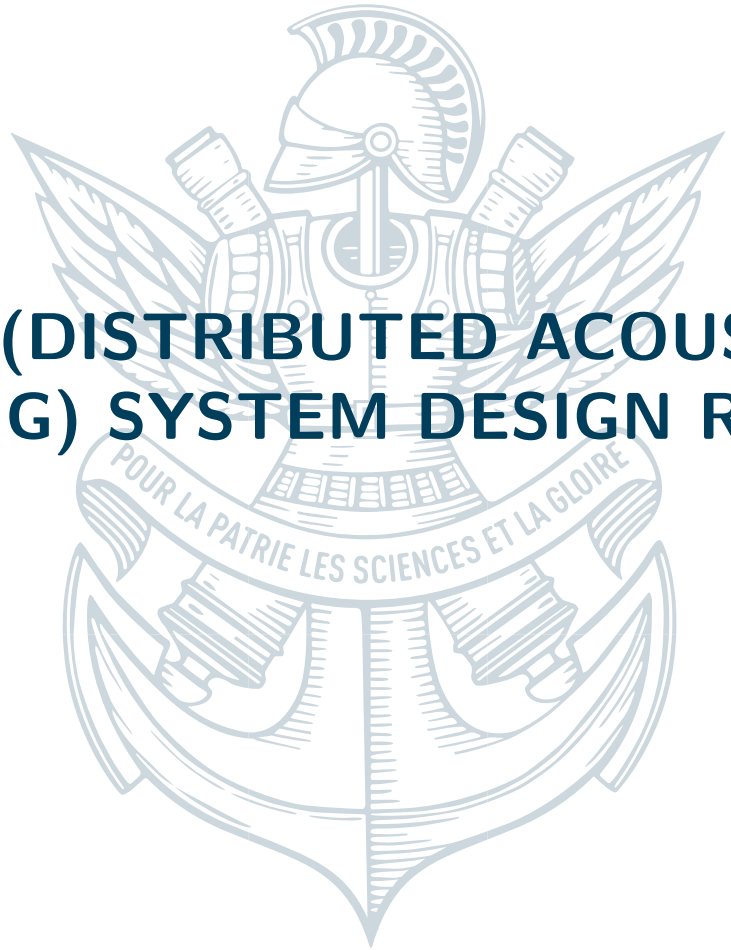

DAS(DISTRIBUTED ACOUSTIC SENSING) SYSTEM DESIGN REPORT

November 10, 2025

Zehua HUANG

1 ABSTRACT

Distributed Acoustic Sensing (DAS) is an emerging optical fiber–based technology that enables real-time, continuous monitoring over long distances. By treating each section of the fiber as a virtual vibration sensor, DAS systems can capture spatially resolved acoustic information with meter-scale precision. This capability makes DAS highly valuable in applications such as perimeter security, pipeline leakage detection, and transportation monitoring.

This internship project, within the theme of digital innovation, focuses on the design and implementation of a high-sensitivity DAS data acquisition, processing, and visualization system. The system adopts a hybrid architecture: C++ is used for high-throughput data acquisition and shared-memory management, while Python provides real-time visualization and an interactive GUI. The main features include:

Zero-copy data sharing between C++ and Python via a custom MemoryBlock and pybind11 interface;

Real-time visualization of waveforms, spectrograms, and waterfall plots with PyQt5 and GPU-accelerated signal processing;

Audio reconstruction of DAS signals for intuitive perception of vibration patterns;

Intelligent anomaly detection and alerting, with event-triggered data backtracking and storage.

The results show that the proposed system improves both the efficiency and usability of DAS data analysis. It contributes scientifically and practically to the digital transformation of distributed sensing technologies.

2 INTRODUCTION

Distributed Acoustic Sensing (DAS) is an advanced technology that uses standard communication optical fibers as sensing media. By launching laser pulses into the fiber and detecting the back-scattered signals, a DAS system can transform a single fiber into thousands of virtual sensing points. This enables real-time, continuous, and distributed monitoring of large-scale environments.

DAS has a wide range of applications: in security, it can be used for intrusion detection and perimeter protection; in the energy sector, for detecting leaks or theft in oil and gas pipelines; and in transportation, for railway and highway vibration monitoring and accident prevention.

Despite its great potential, the practical deployment of DAS faces major challenges, including massive data volumes, strict real-time requirements, and limited user-friendly interfaces. To address these issues, this internship project developed an integrated DAS system for high-performance data acquisition, real-time processing, visualization, and intelligent alerting.

The system is designed to:

1. Reliably acquire and buffer large-scale DAS signals for long-term operation;
2. Provide real-time rendering of waveforms, spectra, and waterfall plots for multi-dimensional visualization;
3. Detect anomalies and generate automatic alerts, with event-triggered data storage and backtracking;
4. Offer an interactive graphical user interface (GUI) to lower the operational threshold and improve user experience.

2.1 System Overview

The DAS system consists of an industrial computer (hosting the control and processing program), a DAS unit (optical/electrical front end), and a sensing optical fiber. The key components and the sequence of operations are described below.

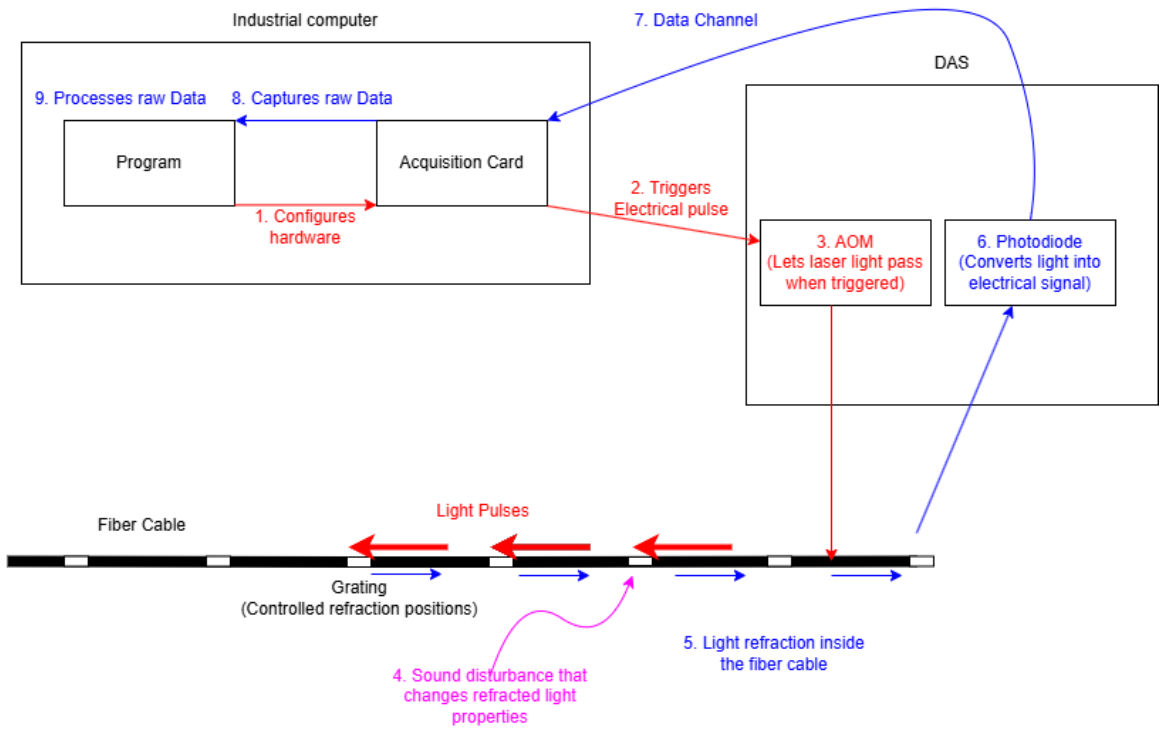

Figure 1: Process of Distributed Acoustic Sensing (DAS) System

- **Industrial Computer:** Contains the control Program and an Acquisition Card responsible for configuring hardware and capturing raw data.
- **DAS Unit:** Includes an Acousto-Optic Modulator (AOM) that gates laser pulses and a photodiode that converts returned optical signals into electrical signals.
- **Optical Fiber:** A sensing fiber with distributed gratings (or natural scattering centers) that reflect part of the injected light back to the DAS unit. External acoustic/vibration events locally perturb the fiber's refractive properties.

The DAS measurement sequence proceeds as follows:

1. By program, user configures the acquisition hardware and measurement parameters (pulse timing, sampling rate, gating, etc.).
2. The acquisition card issues an electrical trigger pulse to the DAS unit.
3. The AOM in the DAS unit is gated by the electrical trigger, allowing short laser pulses to be launched into the sensing fiber.
4. Acoustic or vibrational disturbances along the fiber produce local changes in refractive index (or strain), which in turn alter the phase/intensity of the guided light at those positions.

5. The launched pulses propagate along the fiber and are partially reflected or backscattered by gratings/scattering centers; the backreflected light carries information modulated by the local disturbances.

6. A photodiode converts the returning optical signal into an electrical waveform.

7. The electrical waveform is transmitted through a data channel back to the industrial computer.

8. The acquisition card captures the raw electrical data corresponding to each launched pulse (often storing time-of-flight information to localize events).

9. The **Program** processes the captured raw data to extract acoustic signatures, localize disturbances along the fiber, and perform any required post-processing (filtering, detection, visualization).

2.2 Challenges

In real-world engineering deployments, DAS systems face several major challenges:

1. A single fiber can generate hundreds of MB of data per second-level sampling. Traditional storage and transfer methods cannot handle this scale, making efficient buffering and data-sharing mechanisms necessary.

2. Many applications (e.g., intrusion detection, pipeline leak warning) demand second-level response. Therefore, signal processing algorithms such as FFT and variance detection must be implemented with high efficiency.

3. Limited user experience: Current DAS system only supports offline analysis, which is not convenient for operators.

4. Lack of intelligence: Existing systems often lack real-time alerting and automated decision-making, requiring manual intervention.

This internship addresses these challenges.

2.3 Related Work

This section reviews main technologies in cross-language binding and real-time visualization, and explains the rationale for choosing PyBind11 + PyQt5 + PyQtGraph in this project.

2.3.1 • PyBind11: Efficient and Lightweight C++–Python Binding

Lightweight, header-only library: Unlike Boost.Python, PyBind11 is a header-only library with no external dependencies, which greatly simplifies the build process and reduces deployment complexity. [1]

Modern C++ support: Designed for C++11 and above, PyBind11 provides automatic type conversion (e.g., STL containers directly mapped to Python types), function callbacks, and seamless integration with NumPy arrays. [1]

2.3.2 • PyQt5 + PyQtGraph: Real-Time Visualization Framework

PyQt5 is the Python binding of the Qt framework. It provides a rich set of user interface components, event handling mechanisms, and cross-platform support. With tools like Qt Designer, developers can design interfaces visually and then load them into Python, reducing GUI development complexity and improves maintainability. [2]

PyQtGraph delivers excellent rendering performance built on Qt' s QGraphicsScene architecture, making it suited for dynamic data visualization. It supports OpenGL rendering, significantly improving update speed for high-frequency data streams. It provides features such as zooming, panning, and ROI (Region of Interest) selection, enabling the development of advanced interactive visualization interfaces. [3]

In summary, PyQtGraph is particularly effective for building real-time visualization of signal processing and sensor monitoring applications, while PyQt5 provides the essential framework for GUI construction.

3
FUNCTION DESIGN

3.1 Function Objective

The system is designed to provide a highly sensitive, real-time, and interactive distributed fiber-optic sensing system. Its main functions include:

1. Real-time data acquisition and storage: The system continuously collects and records sensing data. Because it is deployed in an unattended environment, it is engineered for long-term stability and reliable operation.

2. Data visualization: The system converts raw data into graphs and audio, enabling users to understand the results more easily.

3. Anomaly detection and alerting: warn when the real-time data meet the condition.
4. Data management: The system supports automatic and manual data saving and reading.
5. Human-computer interaction: To support users with few programming experiences, the system includes a graphical user interface (GUI), clear comments in the Python code, and clear methods for modifying alert conditions.

3.2 Function Point Description

Table 1 shows the detailed functions and design of the system.

Function Mode	Point	description
Data acquisition and storage	Initialization and calibration	Users configure the device parameters and the system calibrate the grating positions. This step ensures that there is no potential issues, such as fiber or optical module connection errors. If the measured grating position matches the expected configuration (e.g., the interval of grating is 2 m), the system is considered to be operating normally.
Data acquisition and storage	Memory Management	The system allocates an A × 4096 × B memory block in RAM, where data can be written in C++ and read in Python. Here, A means the buffer size defined by the user, 4096 is a fixed frame size determined by the optical module, and B represents the number of gratings, calculated during initialization.
Data acquisition and storage	File Storage	The system provides three modes of data writing: manual writing, automatic sliced writing, and event-triggered writing. In automatic sliced writing, each file is approximately 1 GB, supporting TB-level data circulation. In event-triggered writing, the system records data from three seconds before to three seconds after the warning event, and the duration can be modified by the user.
Data acquisition and storage	Interface Encapsulation	The system provides some Python API (e.g., 'init', 'startRun', 'startWriteFile'), enabling users to develop logic without relying on GUI, which consumes much CPU and GPU resources.
Data visualization	Waveform	It displays the raw data, allowing observation of the sensor acquisition noise.
Data visualization	Spectrogram	It displays signal energy distribution and ambient background noise.
Data visualization	Waterfall Chart (Heatmap)	It displays dynamic energy evolution, showing overall variations along the entire fiber.
Anomaly detection	Real-time Detection, Alert and Saving	The default detection condition is the presence of excessively strong vibrations at any point along the fiber. When this condition is met, the system displays a warning window, shows the timestamps and positions where the threshold is exceeded, and saves the corresponding data. Users can modify the detection condition through a callback function.
Human-computer interaction	GUI	The GUI allows users to save/retrieve data and play real-time reconstructed audio for testing.

Table 1: Function Point Description

3.3 Functional interaction process

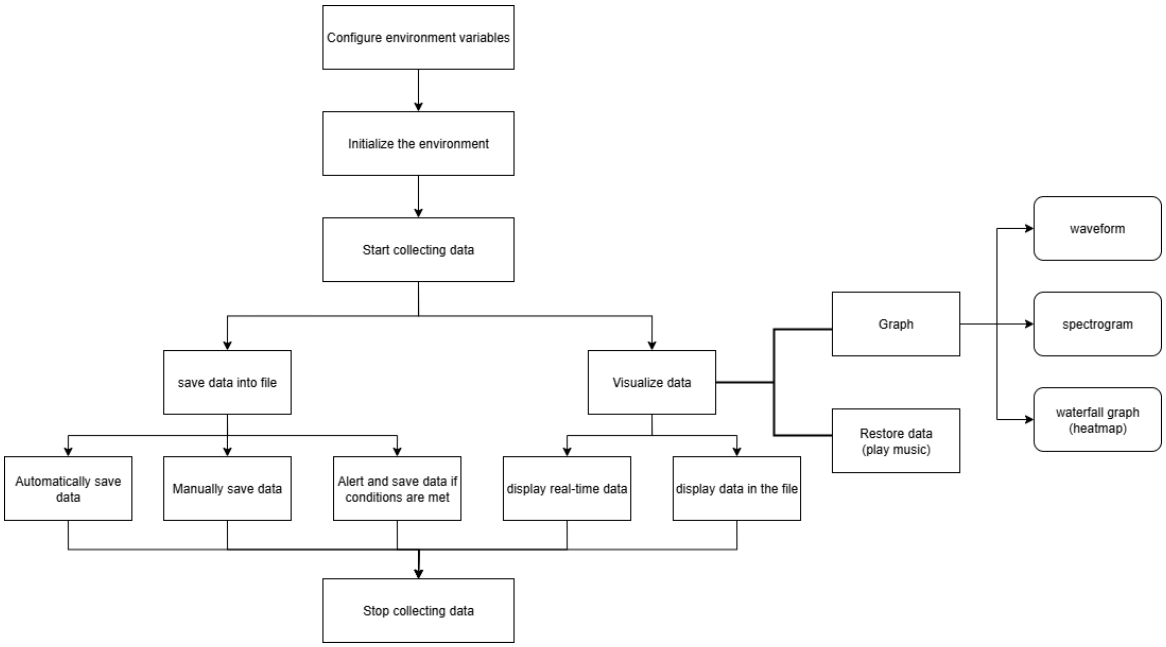

Figure 2: Typical Interaction Flow Chart

Figure 2 presents a typical interaction flow of the system. The process is outlined as follows:

0 *(Optional)* Modify the `conf.ini` file if the operating environment has changed.

1. User runs `GUI.py`, after which the system reads the configuration file and initializes the system.
2. Once initialization is complete, the system automatically generates waveform, spectrogram, and waterfall graphs. Raw data are continuously saved to files.
3. User can select the storage directory by clicking ' 选择目录' (Select Directory). User can save data by clicking ' 开始写入文件' (Start Writing File) and ' 停止写入文件' (Stop Writing File).
4. User can select a specific ' 光栅序号' (Grating Number) to analyze a particular location. The option ' 播放音乐' (Play Music) allows real-time playback of vibrations at the selected location.
5. The user can activate monitoring by clicking ' 开始监视' (Start Monitoring). If an issue is detected, a warning window is displayed showing the affected position, and the corresponding data are saved automatically.
6. Historical data can be read by clicking ' 打开数据' (Read Data).

7. When the user closes the application window, the system terminates the session and saves the log file.

3.4 Graphical User Interface (GUI) Overview

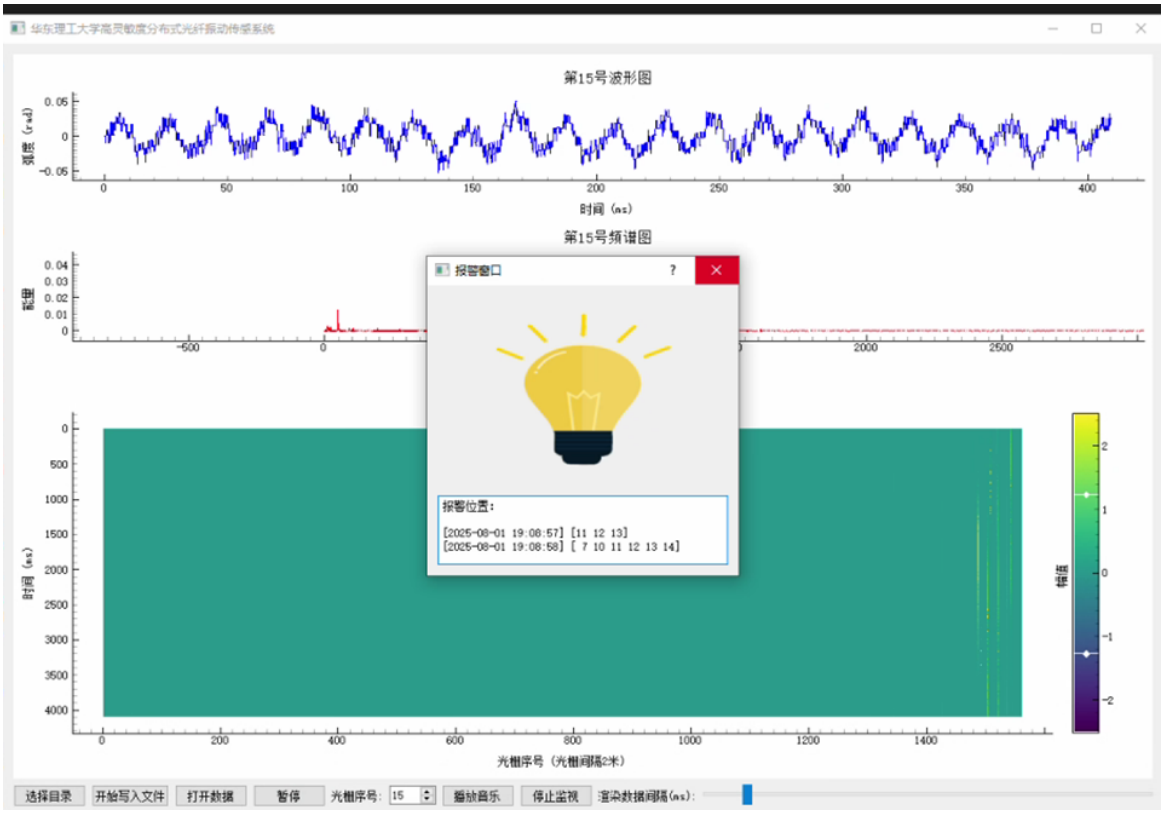

Figure 3: GUI

Figure 3 presents the software interface. The figure shows the visualization panels at the top, including the waveform, spectrogram, and heatmap. The operator controls are at the bottom and include: 'Choose Directory', 'Start Writing Files', 'Read Data', 'Pause', 'Grating Number', 'Play Music', 'Start Monitoring', and 'Data Rendering Interval'. In the middle, an alert window displays the time stamps and the locations of detected events.

4 IMPLEMENTATION

4.1 System and Structure Design

The system architecture is organized into three layers.

4.1.1 • Low-Level Data Collection Layer(C++)

This layer initializes the optical module, configures parameters, and acquires raw data. A custom `MemoryBlock` class is used to manage large memory allocations to ensure stable high-speed data buffering. Encapsulate file writing and an automatic rotation mechanism to ensure continuous recording of TB of data.

4.1.2 • Intermediate Data Transfer Layer (pybind11 interface)

This layer connects C++ and Python through `pybind11`, directly mapping shared memory to `numpy` arrays. It provides a standard API for booting the system and reading and writing files, ensuring zero-copy data transfer and high efficiency.

4.1.3 • Upper Application Layer (Python GUI)

This layer implements real-time visualization and human-computer interaction through `PyQt5` and `pyqtgraph`. It uses `Pyqt5 timers` to implement alarm logic and a normalization algorithm to converse audio.

4.2 Core Algorithm

4.2.1 • Memory Management and Data Sharing (C++ MemoryBlock)

Because the optical module exposes only a C API and requires visual studio environment, developing a GUI directly in C would be complex and inefficient, and Python offers a simpler and more flexible framework for GUI development, the system adopts C++ for data acquisition and Python for visualization.

However, the data rate is extremely high (approximately 100 MB/s under the default configuration). Traditional approaches, such as file-based exchange (C++ → file → Python) or inter-process communication (C++ → pipe → Python), introduce high latency, making real-time performance infeasible.

Proposed solution:

- Shared memory is allocated using a custom C++ `MemoryBlock` class.
- `std::shared_ptr` is used to manage the memory life cycle and prevent memory leaks.
- The shared memory is mapped into a `NumPy` array through `pybind11`, enabling direct access from Python.

Need for matrix transpose: The company's pre-trained machine learning models and signal-processing algorithms require the raw DAS data in a transposed format. Specifically, while the optical module outputs data in a `(time` × `samples` × `channels)` layout, the downstream models expect input arranged in a `(channels` × `time)` or `(features` × `samples)` form. Without this transformation, the models cannot operate correctly. Thus, efficient matrix transpose is not merely an optimization but a *functional requirement* for system compatibility with existing AI pipelines.

Efficient transpose optimization: The company's pre-trained models and signal-processing algorithms require the raw DAS data in a transposed layout. But a naive row-to-column transpose suffers from severe cache misses. To address this, the system employs a *blocked transpose* algorithm with multi-level tiling. Specifically, the matrix is divided into larger blocks that fit into the L2 cache, and each block is further subdivided into smaller tiles optimized for the L1 cache. Within each tile, transposition is performed element-wise, which preserves spatial and temporal locality.

This design is tailored to the cache hierarchy of the Intel Core i7-12700 used in the industrial PC, and improves cache utilization and throughput.

Innovation: This design achieves dual performance benefits:

1. **Zero-copy cross-language data sharing** between C++ and Python, enabling real-time transmission of high-volume DAS signals.
2. **Cache-efficient matrix transpose** via nested blocking, ensuring that the acquired DAS signals can be directly consumed by the company's pre-trained models and algorithms.

4.2.2 • Application Programming Interface (API)

C++ functions (e.g., `Init()` and `startRun()`) are exposed to Python via `pybind11`.

Design approach:

- Performance-critical components, such as memory management and file operations, are implemented in C++.
- Interactivity-critical components, such as alarm logic and user interface control, are implemented in Python.

This hybrid design avoids Python's performance bottlenecks at high-frequency sampling while leveraging the Python ecosystem's strengths in GUIs and numerical computing.

4.3 Visualization Rendering (Python GUI)

The graphical user interface (GUI) integrates three core visualization modules: waveform plots, frequency spectra, and waterfall diagrams. The design objective is to enable real-time monitoring of signal fluctuations, frequency analysis of single-grating signals, and mid-term pattern recognition, while ensuring computational efficiency.

4.3.1 • Waveform Plot (Time-Domain Signal)

The waveform plot provides a direct representation of the real-time signal variation of a single grating. The horizontal axis corresponds to time t (ms), while the vertical axis corresponds to phase (rad).

4.3.2 • Frequency Spectrum (FFT Analysis)

The frequency spectrum enables the identification of characteristic frequency components. In the system, the frequency axis and spectral amplitude are computed using `cupy.fft.fftfreq` and `cupy.fft.fft`, respectively.

4.3.3 • Waterfall Graph (HeatMap)

The waterfall graph provides a two-dimensional perspective for mid- to long-term monitoring and pattern recognition. The horizontal axis corresponds to the grating index, the vertical axis corresponds to time, and the color corresponds to the variance of signal energy. Variance is computed on the GPU via `cupy.var`, and new data segments are appended to the existing waterfall matrix using `cp.concatenate`. This incremental update strategy ensures that only newly acquired rows/columns are processed, to avoiding global recomputation and enabling real-time rendering.

4.3.4 • Key Technical Features

- **GPU Acceleration**: The `cupy` library is employed as a drop-in replacement for `numpy`, significantly accelerating FFT and variance computations;
- **Efficient Rendering**: Heatmaps are rendered using `pyqtgraph.ImageItem`, mitigating the refresh latency typically observed with `matplotlib`;
- **Incremental Updating**: Waterfall data are updated in an incremental manner, with variance computed only for newly acquired data blocks, ensuring both low latency and scalability.

4.4 Anomaly Detection and Warning Mechanism

4.4.1 • Warning Conditions

The system checks for possible anomalies every `check_warn_time` = 1000 ms (1 second). For each check, it uses the most recent few frames of data (default: `dur` = 3 frames). For every grating, the variance of the signal is calculated. If the variance is larger than a preset threshold v_{th}, the system marks that grating as abnormal. When this happens, the system also saves the related data, including the three seconds before and two seconds after the event.

4.4.2 • Warning Window

If an anomaly is found, the GUI opens a `WarnWindow`. This window shows a GIF animation and the time of the warning. At the same time, the system writes a record into a log file for later review.

4.4.3 • Design Rationale

This method was chosen because:

- Variance is a simple and fast way to measure signal energy, making it useful for catching sudden changes.
- The algorithm is intentionally simple to serve as a baseline for users to build their own custom detection methods in the future.
- The short 5-second window (3 seconds before and 2 seconds after the event) gives enough context to understand the situation, but does not take too much storage.

4.5 Signal to Audio

The system can change Distributed Acoustic Sensing signals into audio so that users can directly *listen* to the signals. This is managed by the `AudioPlayer` class, which includes filtering, normalizing, buffering, and playback.

4.5.1 • Processing Steps

The main steps of the algorithm are:

1. **Filtering**: The raw phase signal is passed through a band-pass filter. This removes unwanted noise and keeps only the frequencies we care about.

2. **Normalization**: The signal is scaled so that the maximum amplitude is 1. This prevents the sound from being too loud or distorted:

$$s_{\text{norm}}(n) = \frac{s(n)}{\max |s(n)|}.$$

3. **Buffering and Streaming**: The processed samples are stored in a buffer. When the buffer is filled to a safe level, the system starts playback. New data is continuously added in small blocks so that playback is smooth.

4. **Playback**: The normalized data is sent to the sound card. The playback rate is

$$f_s = \frac{10^6}{\texttt{trigger period}} \quad \text{Hz},$$

which keeps the timing consistent with the DAS system.

4.5.2 • Why This Design

Listening to DAS signals has several benefits:

- Users can audibly perceive vibration events, such as music playback.
- It gives users a natural and intuitive way to understand the data.

5 EXPERIMENTS AND RESULTS

5.1 Experimental Environment

- **Physical environment:** Industrial computer with external speakers; A 100 m black optical fiber is connected to a 5 km bare fiber. Both the speaker and laptop system volume were set to 100%. Figure 4 shows the experimental physical setup.
- **Software environment:** Python 3.8, PyQt5, CuPy, and pybind11. System configuration is managed through the `config.yml` file.
- **Calibration environment:** The complete software environment and configuration files are packaged in `environment+python8.1.zip`. The MD5 hash of this package is `546e9f962313108052ebff708fe638fb`.

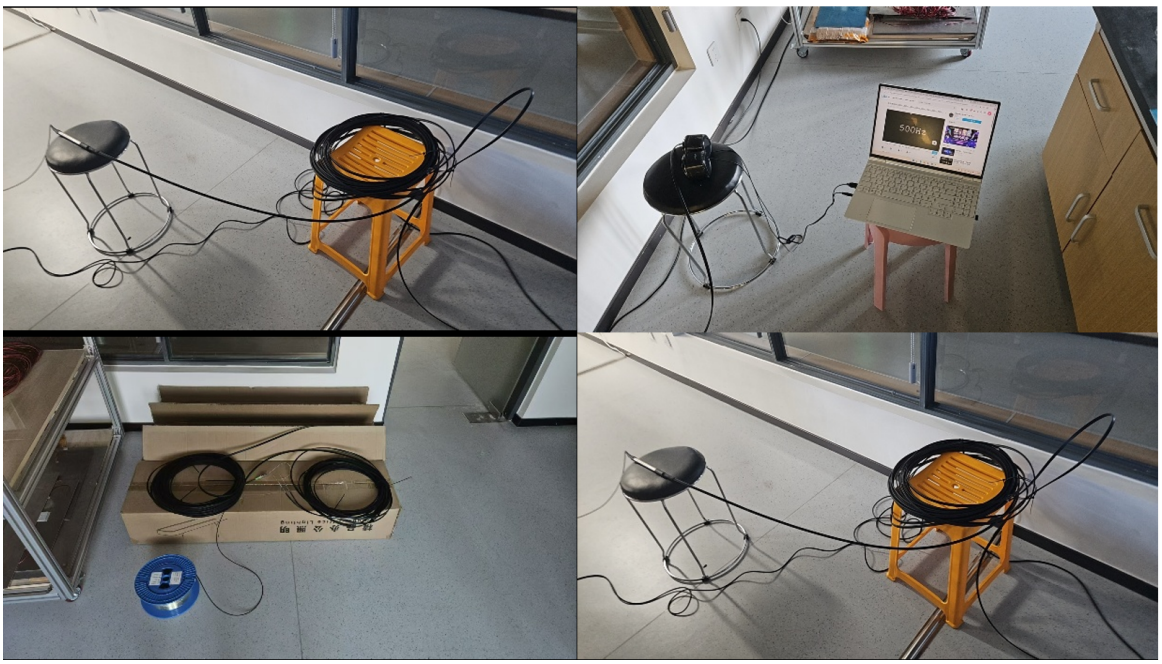

Figure 4: Physical environment

5.2 Experimental Results

- **Spectral response to sinusoidal excitation:** When 100 Hz music[1] was played on a laptop, a clear spectral peak at 100 Hz was observed. Figure 5 shows the spectrum without music playback, while Figure 6 shows the spectrum with 100 Hz music.
- **Audio playback verification:** When the laptop played music, the industrial computer's speaker reproduced the real-time reconstructed signal. Accompaniment, male vocals, and female vocals could be heard clearly. A latency of approximately one second was observed between the laptop playback and the reconstructed audio.
- **Anomaly detection:** When the monitoring function was enabled and an alarm was triggered, a warning window was automatically displayed and the corresponding data were saved. Figure 7 shows the physical vibration applied and Figure 8 presents the saved data and the GUI interface.
- **Data retrieval and replay:** Data reading and replay tests confirmed that the saved logs were consistent with the recorded physical vibrations. Figure **??** shows the example of data read from file. Obviously, there are 4 knocks.
- **Exponential frequency sweep:** Figure 9 shows the results of an exponential frequency sweep. The left panel corresponds to playback through the laptop's built-in speaker,

[1]Source: `https://www.bilibili.com/video/BV1wWEDzyEoR/?share_source=copy_web`

where the loudness was relatively low but the exponential curve was still clearly observed. The right panel corresponds to playback through external speakers, where the loudness was higher. However, due to speaker characteristics, the frequency range 200–400 Hz appeared significantly louder than other regions.

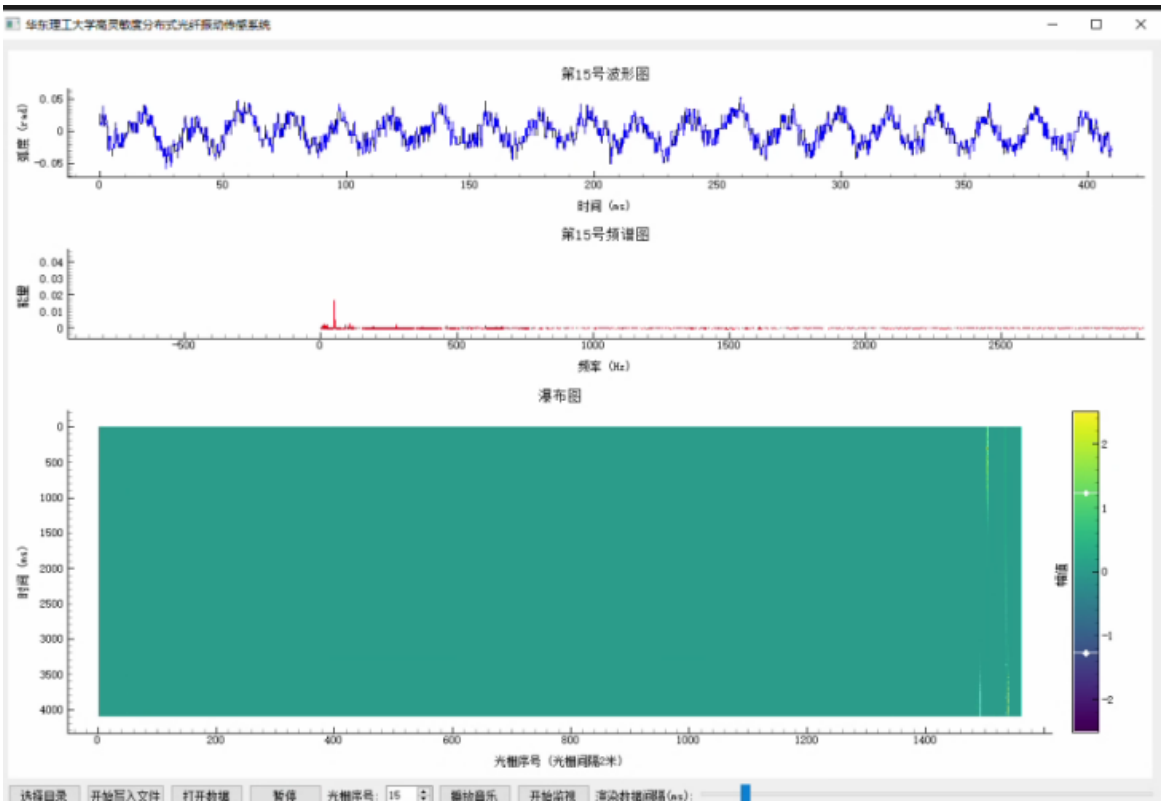

Figure 5: Background Graph

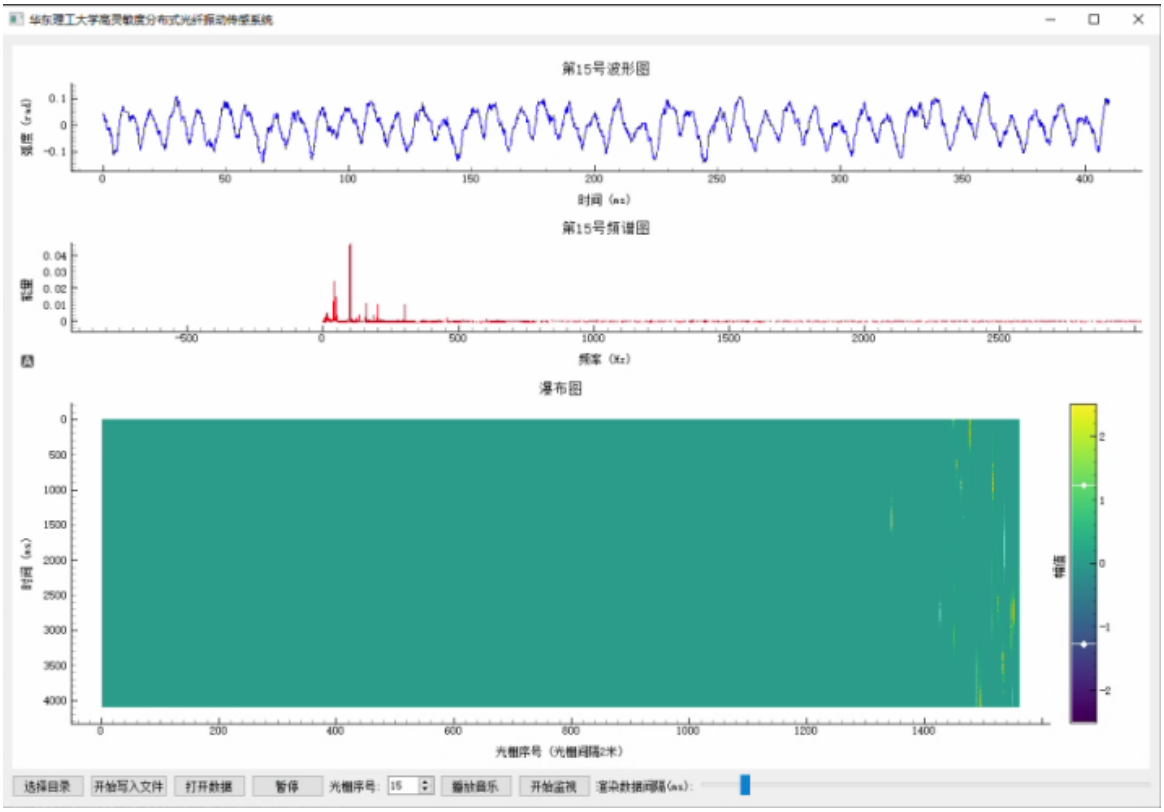

Figure 6: 100 Hz Music Graph

Figure 7: Vibration Applied

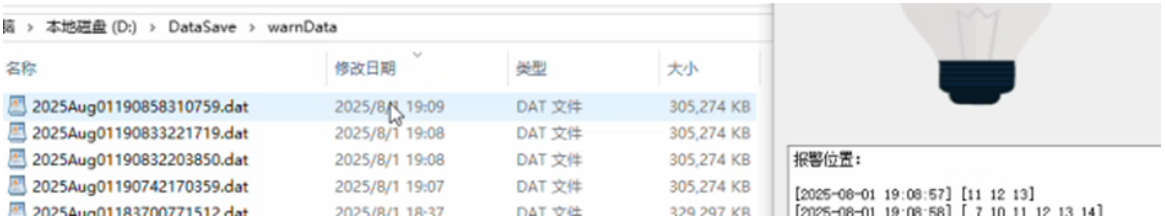

Figure 8: Saved data and GUI

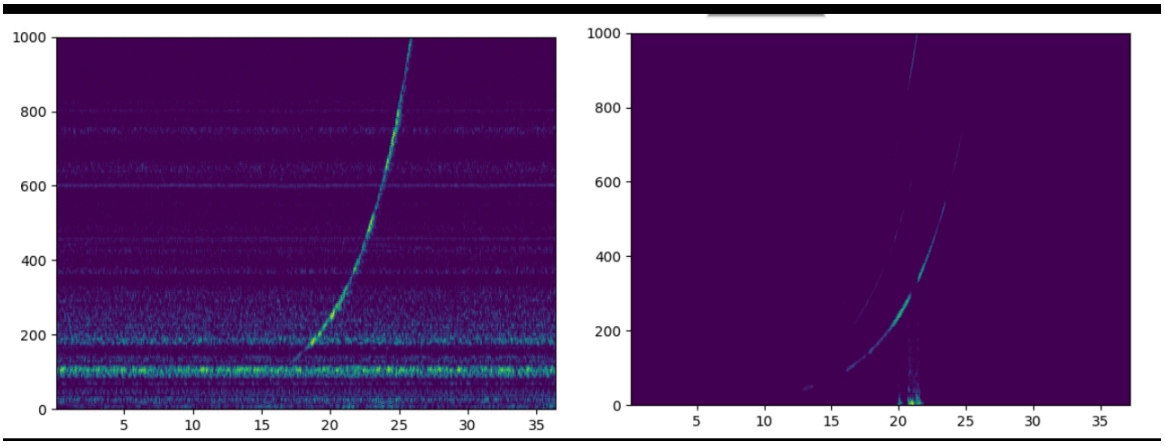

Figure 9: Frequency Sweep Graph

6 REMAINING ISSUES AND FUTURE WORK

Although the system has achieved initial functionality in DAS data acquisition, saving, visualization, and intelligent alerting, there are still several limitations in terms of engineering robustness and intelligent capabilities. Future improvements may focus on the following directions:

1. Code structure and maintainability: Due to unclear requirements in the early stage, the code base is loosely organized. For example, the main.py file is too large, which makes the code difficult to read and maintain. In the future, I plan to refactor the code, separate different functional modules, and build a clearer and more extensible system architecture.

2. Testing convenience: Debugging relies entirely on real optical modules, but the hardware has already been delivered, which limits development and verification. I plan to design a data generator that simulates the output signals of real devices. This will allow software testing and debugging without physical hardware, improving development efficiency and long-term maintainability.

3. Integration of AI models: The system only uses variance-based anomaly detection, without leveraging pre-trained AI models that the company has already developed. I plan to integrate these models into the real-time pipeline and evaluating their performance in operational scenarios, to enhance detection accuracy and the level of system intelligence.

4. Audio–image correspondence: The system can transform fiber signals into audio for playback, and simultaneously produce visualizations such as spectrograms. A research direction is to examine to what extent these visual outputs faithfully represent the characteristics of the corresponding audio signals. I plan to use some methods to compare audio and image features, such as vector similarity, cross-correlation, or short-time Fourier transform (STFT)-based analysis. This would make it possible to objectively measure how well visual outputs capture audio information and lay the groundwork for multi modal analysis. The goal is to ensure semantic consistency between input audio and output images, to improve the practical value of the system.

REFERENCES

[1] pybind11 documentation : https://pybind11.readthedocs.io/en/stable/

[2] https://wiki.python.org/moin/PyQt

[3] https://pyqtgraph.com/is-pyqtgraph-better-than-matplotlib-for-live-plots/?utm_source=chatgpt.com